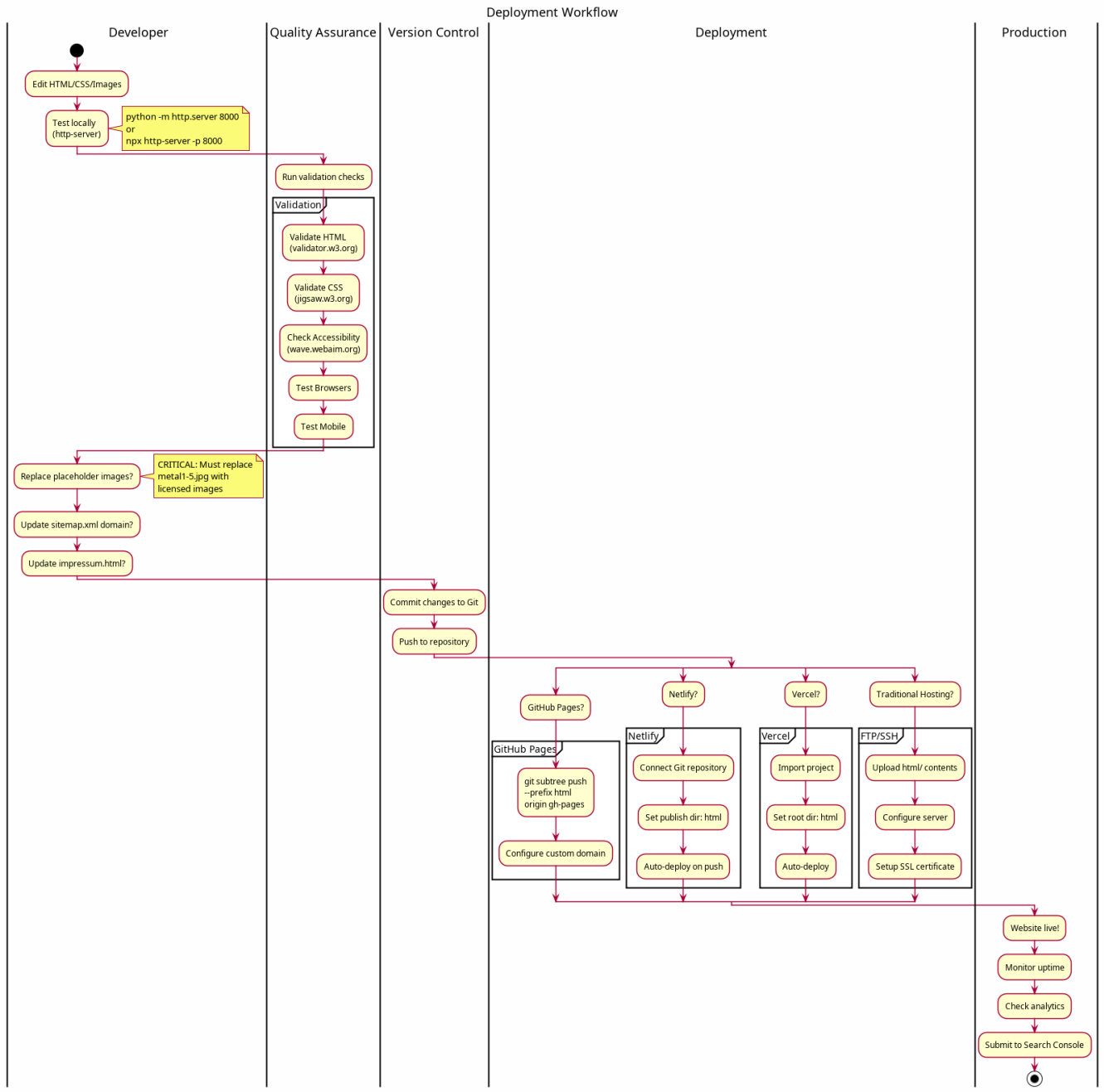
Deployment Guide

Table of Contents

1. Introduction

This guide covers deploying the WebUp Industrial Engineering website to various hosting platforms. Since this is a static HTML/CSS website, deployment is straightforward and works with any static hosting service.



2. Beginner Quick Start: Shared Hosting (FTP / cPanel / Plesk)

If your hosting provider gives you a control panel (like cPanel or Plesk) or FTP/SFTP access, follow these simple steps. No build tools needed — just upload the files.

2.1. 1) Prepare your website

- Replace placeholder images in `html/images/` with your own, properly licensed images
- Fill in your company data in `html/impressum.html`
- Open `html/sitemap.xml` and replace `https://yourdomain.com/` with your real domain
- Optional: Update descriptions and titles in each HTML page

2.2. 2) Locate your web root

On shared hosting, the public web folder is typically named:

- `public_html` (cPanel)
- `httpdocs` (Plesk)
- `www` or `htdocs` (some hosters)

This is the folder your domain points to. Files placed here are served by your website.

2.3. 3) Upload your files

You have two simple options:

1. Using File Manager (in your control panel):
    - Open File Manager and navigate to your web root
    - Upload the CONTENTS of your local `html/` folder (don't upload the `html` folder itself)
    - You should see `index.html`, `about.html`, `css/`, `images/`, etc. directly inside the web root
2. Using an FTP client (FileZilla, Cyberduck, WinSCP):
    - Host: provided by your host (e.g., ftp.yourdomain.com)
    - Protocol: SFTP if available, else FTP
    - Username/Password: from your hoster
    - Remote folder: your web root (e.g., `public_html`)
    - Upload the CONTENTS of your local `html/` folder to the web root

2.4. 4) Test your site

- Visit `https://yourdomain.com`
- You should see your homepage (`index.html`)
- Check a few links (About, Contact, Impressum, image pages)

2.5. 5) Enable HTTPS (SSL)

Most hosters include free HTTPS via Let's Encrypt:

- In cPanel/Plesk: look for "SSL/TLS" or "Let's Encrypt" and enable it for your domain
- After enabling, access the site with `https://` and ensure there are no "mixed content" warnings

2.6. 6) Submit your sitemap (optional, but recommended)

- Go to Google Search Console and add your domain
- Submit `https://yourdomain.com/sitemap.xml`
- Your private pages (about/contact/impressum) are already excluded via meta tags and `robots.txt`

2.7. 7) Common pitfalls (quick fixes)

- Blank page or 404:
  - Ensure `index.html` is in the web root (not inside an extra `html/` folder)
- CSS not loading:
  - Check that `css/style.css` exists in the server's `css/` folder
  - Confirm the `<link rel="stylesheet" href="css/style.css">` path remains relative
- Images not showing:
  - Confirm files are in `images/` and paths match exactly (case-sensitive on Linux)
- Wrong domain in sitemap:
  - Edit `html/sitemap.xml` before uploading (or fix on the server and re-upload)

That's it! You're live with a classic shared hosting setup.

3. Pre-Deployment Checklist

Complete these tasks BEFORE deploying to production:

3.1. Critical Items

- ☐ Replace all placeholder images with licensed images
- ☐ Update `impressum.html` with actual company information
- ☐ Change domain in `sitemap.xml` from `yourdomain.com`
- ☐ Update contact information in `contact.html`
- ☐ Test all internal links
- ☐ Verify all images load correctly

3.2. Quality Assurance

- ☐ Validate HTML at https://validator.w3.org/
- ☐ Validate CSS at https://jigsaw.w3.org/css-validator/
- ☐ Check accessibility at https://wave.webaim.org/
- ☐ Test responsive design on mobile devices
- ☐ Test in multiple browsers (Chrome, Firefox, Safari, Edge)
- ☐ Check page load speed
- ☐ Verify SEO meta tags

3.3. Legal Compliance

- ☐ Confirm image licenses and attributions
- ☐ Review privacy policy (if collecting data)
- ☐ Update copyright year in footer
- ☐ Complete impressum (for German/EU sites)

4. Deployment Options

4.1. GitHub Pages

Free static hosting directly from your GitHub repository.

4.1.1. Prerequisites

- GitHub account
- Repository pushed to GitHub

4.1.2. Deployment Steps

1. **Using gh-pages branch:**

```
# Push only the html folder to gh-pages branch
git subtree push --prefix html origin gh-pages
```

2. **Configure GitHub Pages:**
   - Go to repository Settings → Pages
   - Source: Deploy from branch
   - Branch: `gh-pages` / `root`
   - Click Save
3. **Access your site:**

```
https://yourusername.github.io/webup/
```

4.1.3. Custom Domain

1. Add a `CNAME` file to `html/` folder:

```
www.yourdomain.com
```

2. Configure DNS with your domain provider:

```
Type: CNAME
Name: www
Value: yourusername.github.io
```

3. Add custom domain in GitHub Pages settings

4.2. Netlify

Modern hosting with continuous deployment.

4.2.1. Deployment via Git

1. **Create Netlify account:** https://netlify.com
2. **Connect repository:**
   - Click "New site from Git"
   - Choose GitHub and authorize
   - Select your `webup` repository
3. **Configure build settings:**

```
Base directory: (leave empty)
```

```
Build command: (leave empty - no build needed)
Publish directory: html
```

4. **Deploy:**
   - Click "Deploy site"
   - Netlify will assign a URL like `random-name.netlify.app`

4.2.2. Drag and Drop Deploy

Alternatively, deploy without Git:

1. Zip the `html/` folder contents
2. Go to https://app.netlify.com/drop
3. Drag and drop the zip file
4. Site deploys instantly

4.2.3. Custom Domain

1. Go to Site settings → Domain management
2. Click "Add custom domain"
3. Follow DNS configuration instructions

4.3. Vercel

Fast, global deployment platform.

4.3.1. Deployment Steps

1. **Create Vercel account:** https://vercel.com
2. **Import project:**
   - Click "New Project"
   - Import from GitHub
   - Select `webup` repository
3. **Configure settings:**

```
Framework Preset: Other
Root Directory: html
Output Directory: (leave empty)
Build Command: (leave empty)
```

4. **Deploy:**
   - Click "Deploy"

- Vercel assigns URL like `webup.vercel.app`

4.3.2. Custom Domain

1. Go to Project Settings → Domains
2. Add your domain
3. Configure DNS records as instructed

4.4. Traditional Web Hosting

Deploy to shared hosting, VPS, or dedicated server.

4.4.1. FTP Upload

1. **Connect via FTP client (FileZilla, Cyberduck):**

   ```
   Host: ftp.yourdomain.com
   Username: your_username
   Password: your_password
   Port: 21 (or 22 for SFTP)
   ```

2. **Upload files:**
   - Navigate to `public_html` or `www` directory
   - Upload all files from `html/` folder
   - Maintain directory structure
3. **Set permissions (if needed):**

   ```
   Directories: 755
   Files: 644
   ```

4.4.2. SSH/Command Line

```
# Connect to server
ssh user@yourserver.com

# Create directory
mkdir -p /var/www/yourdomain.com

# Upload files (from local machine)
scp -r html/* user@yourserver.com:/var/www/yourdomain.com/

# Set ownership
sudo chown -R www-data:www-data /var/www/yourdomain.com
```

4.5. Cloud Storage (AWS S3, Google Cloud)

Host as static website on cloud platforms.

4.5.1. AWS S3 Static Website

1. **Create S3 bucket:**

```
aws s3 mb s3://yourdomain.com
```

2. **Upload files:**

```
aws s3 sync html/ s3://yourdomain.com/
```

3. **Enable static website hosting:**
   - Bucket Properties → Static website hosting
   - Index document: `index.html`
   - Error document: `index.html`
4. **Set bucket policy for public access:**

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadGetObject",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::yourdomain.com/*"
  }]
}
```

5. **Access website:**

```
http://yourdomain.com.s3-website-region.amazonaws.com
```

4.5.2. CloudFront CDN (Optional)

Add CloudFront for HTTPS and better performance:

1. Create CloudFront distribution
2. Origin: S3 bucket
3. Enable SSL certificate

4. Update DNS to CloudFront URL

5. Server Configuration

5.1. Apache (.htaccess)

Create `.htaccess` file in `html/` directory:

```
# Security Headers
<IfModule mod_headers.c>
    Header set X-Frame-Options "SAMEORIGIN"
    Header set X-Content-Type-Options "nosniff"
    Header set X-XSS-Protection "1; mode=block"
    Header set Referrer-Policy "strict-origin-when-cross-origin"
    Header set Permissions-Policy "geolocation=(), microphone=(), camera=()"
</IfModule>

# Enable compression
<IfModule mod_deflate.c>
    AddOutputFilterByType DEFLATE text/html text/css text/javascript
application/javascript
    AddOutputFilterByType DEFLATE application/json application/xml
</IfModule>

# Browser caching
<IfModule mod_expires.c>
    ExpiresActive On

    # Images
    ExpiresByType image/jpeg "access plus 1 year"
    ExpiresByType image/png "access plus 1 year"
    ExpiresByType image/x-icon "access plus 1 year"

    # CSS and JavaScript
    ExpiresByType text/css "access plus 1 month"
    ExpiresByType application/javascript "access plus 1 month"

    # HTML
    ExpiresByType text/html "access plus 0 seconds"
</IfModule>

# Force HTTPS
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteCond %{HTTPS} off
    RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
</IfModule>

# Custom error pages
```

```
ErrorDocument 404 /index.html
```

5.2. Nginx Configuration

Add to server block in `/etc/nginx/sites-available/yourdomain`:

```
server {
    listen 80;
    listen [::]:80;
    server_name yourdomain.com www.yourdomain.com;

    # Redirect to HTTPS
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name yourdomain.com www.yourdomain.com;

    # SSL configuration
    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    root /var/www/yourdomain.com;
    index index.html;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Compression
    gzip on;
    gzip_types text/css application/javascript image/svg+xml;
    gzip_min_length 256;

    # Cache control
    location ~* \.(jpg|jpeg|png|gif|ico)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location ~* \.(css|js)$ {
        expires 1M;
        add_header Cache-Control "public";
    }
```

```
    # Main location
    location / {
        try_files $uri $uri/ =404;
    }
}
```

6. SSL/HTTPS Setup

6.1. Let's Encrypt (Free SSL)

For traditional hosting:

```
# Install Certbot
sudo apt update
sudo apt install certbot python3-certbot-nginx

# Get certificate
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com

# Auto-renewal (already set up by Certbot)
sudo certbot renew --dry-run
```

6.2. Cloudflare (Free SSL + CDN)

1. Create Cloudflare account
2. Add your domain
3. Update nameservers at domain registrar
4. Enable SSL in Cloudflare dashboard (Full mode)
5. Configure Page Rules for additional optimization

7. Performance Optimization

7.1. Image Optimization

Before deploying, optimize images:

```
# Install ImageMagick
sudo apt install imagemagick

# Optimize JPEG images
for img in html/images/*.jpg; do
    convert "$img" -quality 85 -strip "$img"
```

```
done

# Optimize PNG images (requires optipng)
sudo apt install optipng
for img in html/images/*.png; do
    optipng -o7 "$img"
done
```

7.2. CSS Minification

Create minified version for production:

```
# Install clean-css-cli
npm install -g clean-css-cli

# Minify CSS
cleancss -o html/css/style.min.css html/css/style.css

# Update HTML to use minified version
```

7.3. Enable Compression

Ensure your server compresses files:

- Apache: Enable `mod_deflate`
- Nginx: Enable `gzip`
- Netlify/Vercel: Automatic

8. Monitoring and Analytics

8.1. Google Analytics

Add before closing `</head>` tag in all HTML files:

```
<!-- Google Analytics -->
<script async
src="https://www.googletagmanager.com/gtag/js?id=GA_MEASUREMENT_ID"></script>
<script>
  window.dataLayer = window.dataLayer || [];
  function gtag(){dataLayer.push(arguments);}
  gtag('js', new Date());
  gtag('config', 'GA_MEASUREMENT_ID');
</script>
```

8.2. Search Console

1. Go to https://search.google.com/search-console
2. Add property for your domain
3. Verify ownership (HTML file upload or meta tag)
4. Submit `sitemap.xml`

9. Troubleshooting

9.1. Common Issues

9.1.1. 404 Errors

Problem: Pages not found after deployment

Solutions:

- Check file paths are relative, not absolute
- Verify all files uploaded correctly
- Check case sensitivity (Linux servers are case-sensitive)
- Ensure `index.html` exists in root

9.1.2. Images Not Loading

Problem: Images don't display on live site

Solutions:

- Verify image files uploaded to `images/` folder
- Check image paths in HTML are correct
- Ensure proper file permissions (644 for files)
- Check browser console for 404 errors

9.1.3. CSS Not Applying

Problem: Styles don't appear on live site

Solutions:

- Clear browser cache
- Check CSS file path in HTML `<link>` tag
- Verify CSS file uploaded correctly
- Check browser console for CSS errors

9.1.4. HTTPS Not Working

Problem: SSL certificate issues

Solutions:

- Verify SSL certificate installed correctly
- Check certificate hasn't expired
- Ensure HTTPS redirect configured
- Test with https://www.ssllabs.com/ssltest/

10. Post-Deployment Tasks

1. **Test live site thoroughly**
2. **Submit sitemap to Google Search Console**
3. **Monitor for 404 errors in analytics**
4. **Set up uptime monitoring (e.g., UptimeRobot)**
5. **Configure backups**
6. **Document deployment process**
7. **Update README with live URL**

11. Continuous Deployment

Set up automatic deployments:

11.1. GitHub Actions

Create `.github/workflows/deploy.yml`:

```
name: Deploy to GitHub Pages

on:
  push:
    branches: [ v0.5.0 ]  # Adjust branch as needed

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Deploy to GitHub Pages
        uses: peaceiris/actions-gh-pages@v3
        with:
```

```
          github_token: ${{ secrets.GITHUB_TOKEN }}
          publish_dir: ./html
```

11.2. Netlify/Vercel

Automatic deployment on Git push (already configured when connecting repository).

12. Rollback Procedures

If deployment fails:

12.1. GitHub Pages

```
# Revert to previous version
git revert HEAD
git push origin gh-pages
```

12.2. Netlify/Vercel

- Use dashboard to rollback to previous deployment
- Select deployment from history
- Click "Publish deploy"

12.3. Traditional Hosting

- Keep backup of previous version
- FTP upload backup files to restore

Last Updated: November 6, 2025
Version: 0.5.0